```
#include <iostream>
#include <algorithm>
#include <iomanip>
using namespace std;
struct Job {
    int id, deadline, profit;
};
bool compare(Job a, Job b) {
    return a.profit > b.profit;
}
void showSlots(int slot[], int maxD) {
    cout << "Time slot: ";
    for (int t = 1; t <= maxD; t++) cout << setw(3) << t;
    cout << endl << "Job id   : ";
    for (int t = 1; t <= maxD; t++) {
        if (slot[t] == 0) cout << setw(3) << "-";
        else cout << setw(3) << slot[t];
    }
    cout << endl;
}
int main() {
    int n;
    cout << "Enter number of jobs: ";
    cin >> n;
    Job jobs[50];
    cout << "Enter job id, deadline and profit for each job:\n";
    for (int i = 0; i < n; i++)
        cin >> jobs[i].id >> jobs[i].deadline >> jobs[i].profit;
    sort(jobs, jobs + n, compare);
    cout << "\nJobs sorted by profit (highest first):\n";
    int maxD = 0;
    for (int i = 0; i < n; i++) {
        cout << "J" << jobs[i].id << ": deadline = " << jobs[i].deadline << ", profit = "
                << jobs[i].profit << endl;
        if (jobs[i].deadline > maxD) maxD = jobs[i].deadline;
    }
    int slot[100];
    for (int i = 0; i < 100; i++) slot[i] = 0;
    int totalProfit = 0;
    for (int i = 0; i < n; i++) {
        cout << "\nStep " << i + 1 << ": Job J" << jobs[i].id << " (deadline = " << jobs[i].deadline
                << ", profit = " << jobs[i].profit << ")\n";
        bool done = false;
        for (int t = jobs[i].deadline; t >= 1; t--) {
            if (slot[t] == 0) {
                slot[t] = jobs[i].id;
                totalProfit += jobs[i].profit;
                cout << "Scheduled in time slot " << t << endl;
                done = true;
                break;
            }
        }
        if (!done) cout << "No free slot on or before the deadline, job rejected\n";
        cout << "Current profit = " << totalProfit << endl;
        showSlots(slot, maxD);
    }
    cout << "\nScheduled jobs: ";
    for (int t = 1; t < 100; t++)
        if (slot[t] != 0) cout << "J" << slot[t] << " ";
    cout << "\nTotal profit = " << totalProfit << endl;
    return 0;
}
```

```
PS C:\Users\fishb\Desktop\Sem 7\DAA> cd "c:\Users\fishb\Desktop\Sem 7\DAA\" ; if ($?) { g++ 2_job_sequencing.cpp -o 2_job
_sequencing } ; if ($?) { .\2_job_sequencing }
Enter job id, deadline and profit for each job:
1 2 20
2 2 15
3 1 10
4 3 5
5 3 1

Jobs sorted by profit (highest first):
J1: deadline = 2, profit = 20
J2: deadline = 2, profit = 15
J3: deadline = 1, profit = 10
J4: deadline = 3, profit = 5
J5: deadline = 3, profit = 1

Step 1: Job J1 (deadline = 2, profit = 20)
Scheduled in time slot 2
Current profit = 20
Time slot:   1  2  3
Job id   :   -  1  -

Step 2: Job J2 (deadline = 2, profit = 15)
Scheduled in time slot 1
Current profit = 35
Time slot:   1  2  3
Job id   :   2  1  -

Step 3: Job J3 (deadline = 1, profit = 10)
No free slot on or before the deadline, job rejected
Current profit = 35
```

```
PS C:\Users\fishb\Desktop\Sem 7\DAA> cd "c:\Users\fishb\Desktop\Sem 7\DAA\" ; if ($?) { g++ 2_job_sequencing.cpp -o 2_job
_sequencing } ; if ($?) { .\2_job_sequencing }
Job id   :   -  1  -

Step 2: Job J2 (deadline = 2, profit = 15)
Scheduled in time slot 1
Current profit = 35
Time slot:   1  2  3
Job id   :   2  1  -

Step 3: Job J3 (deadline = 1, profit = 10)
No free slot on or before the deadline, job rejected
Current profit = 35
Time slot:   1  2  3
Job id   :   2  1  -

Step 4: Job J4 (deadline = 3, profit = 5)
Scheduled in time slot 3
Current profit = 40
Time slot:   1  2  3
Job id   :   2  1  4

Step 5: Job J5 (deadline = 3, profit = 1)
No free slot on or before the deadline, job rejected
Current profit = 40
Time slot:   1  2  3
Job id   :   2  1  4

Scheduled jobs: J2 J1 J4
Total profit = 40
PS C:\Users\fishb\Desktop\Sem 7\DAA>
```